

Лабораторна робота №1

Тема: Строга типізація у вебi: екосистема мови TypeScript.

Мета: ознайомлення з основними можливостями та синтаксичними конструкціями мови програмування TypeScript; набуття практичних навичок використання транслятора TypeScript та конфігураційного файлу tsconfig.json; формування вмінь застосування статичної типізації змінних і функцій; вивчення принципів об'єктно-орієнтованого програмування в TypeScript, зокрема створення та використання класів, конструкторів, модифікаторів доступу, властивостей із використанням конструкцій get і set; опанування механізмів успадкування та абстрактних класів; ознайомлення з інтерфейсами та набуття навичок їх застосування під час опису функцій і класів.

Вимоги до звіту:

1. Дотримання TypeScript-типізації
2. Кожне завдання має бути виконано в окремому ts файлі
3. Кожне завдання - окремий commit з відповідним commit message
4. Код має бути читабельним і добре організованим
5. Код повинен бути розміщений в особистому GitHub репозиторії студента

Теоретична частина

1. TypeScript for the New Programmer <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html>
2. TypeScript for JavaScript Programmers <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>
3. TypeScript Tooling in 5 minutes <https://www.typescriptlang.org/docs/handbook/typescript-tooling-in-5-minutes.html>
4. TypeScript Basics <https://www.typescriptlang.org/docs/handbook/2/basic-types.html>
5. What is a tsconfig.json <https://www.typescriptlang.org/docs/handbook/tsconfig-json.html>
6. nodejs <https://nodejs.org/en>
7. package.json <https://docs.npmjs.com/cli/v10/configuring-npm/package-json>
8. Functions - <https://www.typescriptlang.org/docs/handbook/2/functions.html#handbook-content>
9. JavaScript Classes - <https://uk.javascript.info/classes>
10. Classes - <https://www.typescriptlang.org/docs/handbook/2/classes.html>
11. Object Types - <https://www.typescriptlang.org/docs/handbook/2/objects.html>

Практична частина

Завдання 1.

1. Вивчити основні поняття, розглянуті під час лекції.
2. Встановіть Node.js, якщо це ще не зроблено.
3. Створіть новий проєкт, використовуючи `npm init` для ініціалізації `package.json`.
4. Встановіть TypeScript глобально та локально у проєкт.
5. Ініціалізуйте TypeScript у вашому проєкті.
6. Проведіть налаштування компілятора TypeScript.

Завдання 2.

1. Створіть файл `variables.ts`. Оголосіть змінні різних типів (рядок, число, булевий) і ініціалізуйте їх значеннями. Використовуйте типи `string`, `number`, `boolean`, `any`. Виведіть значення всіх змінних у консоль.
2. Створіть масиви для рядків і чисел. Оголосіть масиви з конкретними типами елементів.
3. Скомпілюйте файл `variables.ts` і перевірте результати у консолі.
4. Створіть функцію, яка приймає рядок і число, з параметром за замовчуванням для числа. Якщо число не передано, функція повинна використовувати значення за замовчуванням. Скомпілюйте файл `functions.ts` і перевірте функції у консолі.

Завдання 3.

Використовуючи TypeScript, створіть наступну функцію, яка розраховує вартість морозива. Магазин пропонує 2 розміри морозива:

- Маленький стаканчик (10 грн)
- Великий стаканчик (25 грн)

Морозиво доповнюється однією або декількома начинками (мінімум – однією):

- шоколад (+5 грн)
- карамель (+6 грн)
- ягоди (+10 грн) Додатково морозиво (не обов'язково) можна посипати маршмелоу (+5 грн).

Вхідні параметри користувач вводить через `prompt`.

Завдання 4.

1. Створіть інтерфейс, що описує поведінку тварини (властивості, способи пересування).
2. Використовуйте цей інтерфейс до класу `Cat`, `Bird`, `Fish`.

3. Подумайте, які властивості мають бути опціональними.

Завдання 5.

1. Створіть інтерфейс Shape, який міститиме методи для обчислення площі (getArea(): number) та периметру (getPerimeter(): number).
2. Реалізуйте класи Circle, Rectangle, та Triangle, які імплементуватимуть інтерфейс Shape. У кожному класі повинні бути відповідні властивості для збереження розмірів фігури.
3. Додайте до кожного класу метод для масштабування фігури (scale(factor: number)), який буде змінювати розміри фігури пропорційно до заданого коефіцієнта.
4. Напишіть код, який створює масив об'єктів типу Shape, додаючи в нього різні фігури, та обчислює загальну площу та периметр усіх фігур у масив

Завдання 6.

1. Створити поняття абстрактного батьківського класу Car. Від нього створити 3 похідні класи (марки автомобілів) із застосуванням методу super().
2. У класах використовувати модифікатори як у батьківському класі, і у похідних.
3. Створити від похідних класів мінімум по 2 екземпляри (моделі автомобілів). Методи у похідних класах повинні виводити на екран усі властивості (опис автомобіля).
4. Подумайте, які властивості у похідних класах мають бути public, які – private та protected.

Завдання 7.

1. Створіть абстрактний клас Employee, який міститиме поля для імені, віку та заробітної плати. Додайте абстрактний метод getAnnualBonus(): number, який повертатиме річний бонус співробітника.
2. Реалізуйте класи Developer та Manager, що наслідуватимуть клас Employee. Клас Developer має повертати бонус у розмірі 10% від зарплати, а клас Manager – 20%.
3. Додайте інтерфейс Payable, який міститиме метод pay(): void. Обидва класи Developer та Manager повинні імплементувати цей інтерфейс.
4. Напишіть код для створення масиву об'єктів типу Employee, який міститиме розробників і менеджерів, та підраховує загальну річну суму бонусів для всіх співробітників.

Завдання 8.

1. Створіть інтерфейс Course, який міститиме поля для назви курсу, тривалості (в годинах), та масиву студентів.
2. Реалізуйте клас OnlineCourse, який імплементуватиме інтерфейс Course. Додайте метод для реєстрації студентів на курс (registerStudent(student: string): void) та метод для

перевірки, чи студент вже зареєстрований на курс (`isStudentRegistered(student: string): boolean`).

3. Створіть клас `CourseManager`, який буде керувати масивом курсів. Додайте методи для додавання курсу (`addCourse(course: Course): void`), видалення курсу (`removeCourse(courseName: string): void`), та пошуку курсу за назвою (`findCourse(courseName: string): Course | undefined`).
4. Напишіть код, який створить кілька курсів, додасть їх до `CourseManager`, зареєструє студентів на деякі курси, та виведе список курсів разом із зареєстрованими студентами.

Завдання 9.

1. Створіть інтерфейс `LibraryItem`, який міститиме властивості для назви та автора. Додайте метод `borrow(): void`, який позначатиме елемент як позичений.
2. Реалізуйте класи `Book`, `Magazine`, та `DVD`, які будуть імплементувати інтерфейс `LibraryItem`. Кожен клас повинен мати специфічні для нього властивості (наприклад, для книг – кількість сторінок, для журналів – номер випуску, для DVD – тривалість).
3. Створіть клас `Library`, який буде містити масив бібліотечних елементів. Додайте методи для додавання елементів до бібліотеки (`addItem(item: LibraryItem): void`), пошуку елементів за назвою (`findItemByName(name: string): LibraryItem | undefined`), та виведення списку доступних елементів.
4. Напишіть код, який демонструє роботу бібліотеки, додаючи до неї різні елементи та виконуючи операції з ними.

Контрольні запитання:

1. Що таке TypeScript і які його основні переваги порівняно з JavaScript?
2. Як налаштувати середовище для розробки TypeScript? Назвіть основні кроки.
3. Що таке файл `tsconfig.json` і яку роль він відіграє в проєкті TypeScript?
4. Які основні типи даних підтримує TypeScript? Дайте короткий опис кожного типу.
5. Чим відрізняється `let` від `var`?
6. Що таке тип `any` і коли його слід використовувати?
7. Як можна визначити власний тип даних у TypeScript? Наведіть приклад.
8. Як компілювати файли TypeScript у JavaScript? Яка команда для цього використовується?
9. Що таке `--watch` режим компілятора TypeScript і як він може бути корисний?
10. Які опції можна вказати у файлі `tsconfig.json` для налаштування процесу компіляції? Назвіть принаймні три з них.
11. Для чого застосовуються `map`-файли?
12. Яким чином можна оголосити змінну з конкретним типом даних у TypeScript? Наведіть приклад.
13. Що таке типи масивів у TypeScript і як їх правильно оголосити?

14. Які основні відмінності між типами `number` і `any` в TypeScript?
15. Як оголосити функцію у TypeScript з типами параметрів та типом повернення? Наведіть приклад.
16. Що таке параметри за замовчуванням у функціях TypeScript і як їх використовувати?
17. Що таке функція, яка повертає `void`, і в яких випадках її слід використовувати?
18. Для чого використовується метод `call`, `apply`?
19. Як у TypeScript реалізовано наслідування?
20. Що таке абстрактний клас? Абстрактний метод?
21. Навіщо використовується ключове слово `implements`?
22. У чому різниця між `private` і `protected`?
23. Навіщо використовується ключове слово `static`? `readonly`?
24. Що таке опціональні параметри в інтерфейсі?
25. Навіщо використовується конструктор у класа?